Kubernetes-Native ICAP Operator for Adaptive Malware Inspection Infrastructure Management

Senali Amanda Lelwala Guruge

IT22353634

Dissertation submitted in partial fulfilment of the requirements for the

Bachelor of Science (Hons) in Information Technology

Specialising in Cyber Security

Department of Computer Systems and Engineering

Sri Lanka Institute of Information Technology

April 2026

DECLARATION

I declare that this is my own work, and this dissertation does not incorporate without acknowledgement of any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation in whole or part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as article or books).

Name	Student ID	Signature
S A L Guruge	IT22353634	

Supervisor: Mr. Amila Senarathne

Signature: _____ Date: _____

ABSTRACT

Kubernetes has become the dominant platform for containerized workload orchestration, yet the security tooling available for inline malware inspection within cluster boundaries remains largely manual, static, and unaware of the dynamic scaling demands inherent to security-sensitive workloads. Traditional deployments of ClamAV and the Internet Content Adaptation Protocol (ICAP) are configured at the network edge, leaving intra-cluster traffic uninspected. Furthermore, the Kubernetes Horizontal Pod Autoscaler scales security workloads on CPU and memory metrics that are blind to the signals which actually determine malware detection reliability.

This dissertation presents the ICAP Operator, a Kubernetes-native controller built with Kubebuilder that automates the full lifecycle of ICAP and ClamAV scanning services through a Custom Resource Definition (CRD). The operator introduces a dynamic health scoring mechanism which produces a composite score from four security-relevant metrics scan latency, error rate, backlog depth, and signature freshness and drives adaptive autoscaling through KEDA, replacing CPU-only scaling with a security aware policy.

Evaluated on a three-node Minikube cluster, the operator achieved pod self-healing in 28.4 seconds, zero dropped scans during ClamAV signature hot-reload, a health score of 94/100 under all-healthy conditions, a 22.4 percent resource efficiency improvement over CPU-only autoscaling, and a 100 percent EICAR detection rate with zero false positives across 3,000 test scans. All twelve benchmark criteria produced PASS results.

The work establishes a reusable pattern applicable to any Kubernetes security workload: define security-relevant health metrics, expose them as a Prometheus gauge, and connect to KEDA for calibrated autoscaling.

Keywords: *Kubernetes, ICAP, ClamAV, Kubebuilder, adaptive autoscaling, KEDA, health scoring, cloud-native security.*

ACKNOWLEDGEMENT

The author wishes to express sincere gratitude to Supervisor Mr. Amila Senarathne and Co-Supervisor Mr. Kavinga Yapa Abeywardena of the Department of Computer Systems and Engineering, Sri Lanka Institute of Information Technology, for their continuous guidance, critical feedback, and encouragement throughout this research. Their expertise in distributed systems, cloud-native architecture, and security research was invaluable in shaping the direction and rigour of this dissertation.

The author also thanks the CAPSLock project group members, Marlon Deashapriya, Kaavya Raigambandarage and Themiya Pandhukabaya for their close collaboration in establishing the architectural boundaries, integration contracts, and testing interfaces that shaped the ICAP Operator's design.

Special thanks are due to the open-source communities behind Kubernetes, Kubebuilder, ClamAV, c-icap, Prometheus, KEDA, and Istio, whose publicly available source code, documentation, and community forums made this research feasible.

Finally, the author acknowledges the support of family and colleagues during the extended development and writing periods, and the Department of Computer Systems and Engineering for providing access to the laboratory infrastructure used for experimental evaluation.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AKS	Azure Kubernetes Service
ClamAV	Clam AntiVirus — open-source signature-based antivirus scanner
CRD	Custom Resource Definition
DXA	Device-independent eXtra-fine Absolute unit (Word measurement)
eBPF	Extended Berkeley Packet Filter
EAPE	Environment-Aware Policy Engine (CAPSLock component 2)
HPA	Horizontal Pod Autoscaler
ICAP	Internet Content Adaptation Protocol (IETF RFC 3507)
K8s	Kubernetes
KEDA	Kubernetes Event-Driven Autoscaling
MAC	Mandatory Access Control
MEDS	Multi-Environment Deployment System (CAPSLock component 1)
mTLS	Mutual Transport Layer Security
RBAC	Role-Based Access Control
REQMOD	Request Modification mode of the ICAP protocol
SDLB	Service Discovery and Load Balancing (CAPSLock component 3)
SIGHUP	Signal Hang-Up — POSIX signal used to reload daemon configuration

1 INTRODUCTION

This thesis focuses on strengthening security within Kubernetes based environments where containerized applications communicate dynamically and on a scale. As organizations increasingly adopt cloud native architectures, traditional security mechanisms that operate at the network perimeter are no longer sufficient to protect internal cluster traffic. Malware inspection systems such as ICAP and ClamAV are typically deployed in static configurations at the edge, making them ineffective for monitoring east west traffic between microservices. At the same time, existing autoscaling approaches rely on resource utilization metrics that do not reflect the actual health or reliability of security workloads. This creates a critical need for intelligent, adaptive mechanisms that can both maintain strong malware detection capabilities and efficiently manage system resources. In this context, this research proposes a Kubernetes native ICAP Operator that integrates lifecycle automation, dynamic health scoring, and adaptive autoscaling. By leveraging real time security relevant metrics such as scan latency, error rate, backlog depth, and signature freshness, the system is designed to continuously evaluate the effectiveness of the scanning infrastructure and respond proactively to changes in workload conditions.

This introduction is organized into four key sections. The first section presents the background and literature survey, situating this work within existing research on Kubernetes, ICAP based malware detection, and autoscaling mechanisms. The second section defines the research gap by identifying limitations in current approaches to security workload management in Kubernetes environment. The third section formulates the research problem addressed by this study. Finally, the fourth section outlines the research objectives, describing the goals and expected outcomes of the proposed solution.

1.1 Background Literature

1.1.1 Container orchestration and kubernetes

Container orchestration has fundamentally transformed the operational model of modern distributed systems. Kubernetes, released by Google in 2014 and subsequently donated to the Cloud Native Computing Foundation (CNCF), has become the dominant substrate for containerized workload management [1]. Its declarative object model, controller-based reconciliation loop, pluggable networking layer, and extensive ecosystem of operators and extensions have produced an environment that is both highly automated and highly dynamic with

workloads arriving and departing at rates that would have been operationally unmanageable under traditional virtualization.

The scale of Kubernetes adoption makes its security posture a matter of significant concern. A 2023 Red Hat survey found that 67 percent of Kubernetes users had delayed or slowed adoption specifically due to security concerns, and 37 percent had experienced a security incident in their Kubernetes environment in the preceding twelve months. Containers share the host kernel, which means that a vulnerability in a container runtime or a misconfigured workload can expose the entire cluster to privilege escalation and lateral movement [2]. Large-scale empirical studies confirm that a significant proportion of publicly available container images contain outdated packages and high-severity vulnerabilities, and that most Kubernetes misconfigurations arise from security-unaware default configurations rather than deliberate choices [3].

The CAPSLock project was conceived to address the gap between Kubernetes's automation capabilities and the manual, static nature of the security controls that organizations typically deploy around it. The platform comprises four components: the Multi-Environment Deployment System (MEDS, component 1), the Environment-Aware Policy Engine (EAPE, component 2), the Service Discovery and Load Balancer (SDLB, component 3), and the ICAP Operator (component 4 - the subject of this dissertation). Within this architecture, the ICAP Operator is the deepest enforcement layer: every deployment that passes through MEDS, is scored by EAPE, and is routed by SDLB ultimately depends on the ICAP Operator to keep the malware-scanning infrastructure alive, healthy, and correctly sized.

1.1.2 ICAP protocol and ClamAV malware detection

The Internet Content Adaptation Protocol (ICAP) was standardized in IETF RFC 3507 by Elson and Cerpa [4] as a lightweight HTTP-like protocol for executing remote procedure calls on HTTP messages. Two primary interaction modes are defined: REQMOD (Request Modification) for request-side adaptation antivirus scanning, URL filtering, authentication injection and RESPMOD (Response Modification) for response-side adaptation DLP, content filtering, and data transformation. ICAP is well established in enterprise proxy and gateway architecture; Symantec, McAfee, Sophos, and F5 all implement ICAP client interfaces.

ClamAV, developed by Sourcefire and now maintained by Cisco, is the most widely deployed open-source antivirus engine. It provides signature-based detection using virus definition files

(main.cvd, daily.cvd) updated by the freshclam utility. Ilca and Lucian [5] demonstrated the integration of ICAP with ClamAV in an OPNsense-based SME gateway, confirming that ICAP-based scanning offloads content inspection from client endpoints without requiring local antivirus software. Mangipudi et al. [6] demonstrated hardware-accelerated ClamAV achieving real-time detection on high-throughput links. Wang and Lai [7] showed that ICAP-based inspection is feasible even in resource-constrained IoT gateway deployments.

Despite the maturity of ICAP and ClamAV individually, no prior published system has integrated ICAP-based malware scanning into the Kubernetes lifecycle management and service discovery layer. Existing deployments are static, manually configured at the network edge, and structurally unable to inspect intra-cluster traffic, traffic between microservices running within the same cluster never reaches an edge scanner. This gap is the primary motivation for the ICAP Operator.

1.1.3 Kubernetes operators and lifecycle automation

The Kubernetes Operator pattern was formalized by Burns, Beda, and Grant [8] as an extension mechanism that encodes domain-specific operational logic into controllers and Custom Resource Definitions (CRDs), allowing Kubernetes to manage complex stateful services with the same automation it provides for stateless workloads. Hausenblas and Schimanski [9] and Dobies and Wood [10] subsequently documented the operational patterns that reliable operators follow: the reconciliation loop, informer-based watches, work-queue duplication, and idempotent reconcile functions.

Operators have been successfully applied across a wide range of domains. Shenoy, Rathore, and Jha [11] demonstrated the Operator pattern applied to observability frameworks, automating the lifecycle of Prometheus, Alertmanager, and Grafana stacks. Trapezin and Filianin [12] applied it to machine-learning workflow management in Kubernetes. However, Xu, Liu, and Zhou [13] provide the most sobering assessment: a comprehensive empirical study of bugs in 60 production Kubernetes Operators, finding that operators are frequently buggy and difficult to maintain. The three most common failure modes are: (i) incorrect reconciliation termination conditions causing infinite loops; (ii) missing finalizer logic leaving orphaned resources after CRD deletion; and (iii) status sub-resource inconsistency, where the reported state diverges from the actual cluster state. The ICAP Operator was designed with explicit mitigations for all three failure modes.

1.1.4 Autoscaling for Security Workloads

The Kubernetes Horizontal Pod Autoscaler (HPA) has well-documented limitations when applied to security workloads. Molleti [14] provides the most operationally relevant analysis, demonstrating that CPU and memory metrics fail to capture the health of a ClamAV-based scanning service under realistic threat patterns. A ClamAV pod that is scanning files with a 48-hour-old signature database will exhibit CPU utilization indistinguishable from a pod scanning with a current database, yet the former poses a significant detection risk.

Nguyen et al. [15] extended this analysis empirically, showing that HPA-managed replicas serving content-inspection workloads exhibit systematic under-provisioning when backlog depth and error rate are the dominant demand signals. Augustyn, De Souza, and Martins [16] proposed statistical tuning of HPA parameters as a partial mitigation but noted that parameter optimization cannot substitute for metric selection: no amount of threshold tuning makes CPU a reliable proxy for scan throughput under variable payload sizes.

KEDA (Kubernetes Event-Driven Autoscaling) provides the infrastructure for custom-metric-driven scaling that the ICAP Operator exploits. KEDA can ingest any Prometheus metric as a scaling trigger via its Prometheus Scaled Object scaler [17]. This makes it the natural integration target for the health score produced by the ICAP Operator's health monitoring subsystem. Guruge and Priyadarshana [18] demonstrated time-series-based Kubernetes autoscaling using LSTM and Facebook Prophet models, achieving accuracy improvements of 18–24% over reactive HPA motivating the combination of real-time health scoring and KEDA-driven scaling adopted in this dissertation.

1.1.5 Container Security: Threats and Mitigations

Kang, Lee, and Choi [19] demonstrated that eBPF-based kernel exploits can bypass container isolation in configurations that pass standard admission checks, arguing for in-cluster runtime enforcement rather than perimeter-only controls. Hwang and Park [20] evaluated Mandatory Access Control frameworks, SELinux and AppArmor, in Kubernetes, finding that MAC-level isolation significantly reduces the attack surface of the shared kernel. Singh, Kumar, and Gupta [21] applied STRIDE threat modelling to containerised environments, identifying supply-chain, registry, and orchestrator weaknesses as primary risk categories, all of which are within the threat scope that ICAP-based scanning is designed to address.

Yang, Zhou, and Zhang [22] demonstrated provenance-based detection of malicious activity in container environments, and Kim and Cho [23] analysed cross-container kernel exploits in Kubernetes clusters. Morabito and Kim [24] surveyed container runtime security vulnerabilities, proposing hardening solutions including user namespace enablement and regular patching, measures that are complementary to, rather than substitutes for, inline content inspection.

1.2 Research Gap

A synthesis of the literature reveals three specific gaps that motivate the ICAP Operator component of the CAPSLock platform. Each gap represents an absence in the published literature and in available open-source tooling.

Gap 1 - No Kubernetes-native ICAP lifecycle operator: No prior published system or open-source project has implemented a Kubernetes Operator for ICAP-based ClamAV lifecycle management. Table 1.1 (presented in the Methodology chapter) documents this gap systematically. ICAP deployments remain static and manually configured at the network edge, with no automation of pod creation, scaling, signature updates, or health monitoring. This absence means that organisations running Kubernetes in production must maintain ICAP scanning infrastructure through manual procedures that are error-prone, operationally expensive, and scale-unaware.

Gap 2 - No security-relevant health scoring for Kubernetes autoscaling: The Kubernetes autoscaling literature contains no published composite health score derived exclusively from ICAP-layer security metrics. Existing autoscaling research addresses performance workloads (web servers, microservices, databases) using resource utilisation signals. The four signals most relevant to ICAP workload health, scan latency, error rate, backlog depth, and ClamAV signature freshness, have not been combined into a Kubernetes-actionable health score in any prior work. The absence of such a score means that autoscaling decisions for security workloads are systematically miscalibrated.

Gap 3 - No zero-downtime signature management pattern for in-cluster ClamAV: Keeping ClamAV signatures current is essential for detection efficacy, but hot-reloading signatures without pod restart, without service interruption, and without dropping in-flight ICAP scans is a non-trivial operational problem. The existing operator literature (including Xu et al. [13]) does not address this challenge in a Kubernetes context. Existing signature update strategies either restart pods

(dropping in-flight scans) or operate outside the Kubernetes lifecycle (manual cron jobs on the host).

1.3 Research Problem

Kubernetes has become the most popular platform for containerised application orchestration, yet the integration of robust, automated malware inspection into the Kubernetes lifecycle management framework remains an unsolved problem. The two root causes identified in the literature are: (i) the absence of a Kubernetes-native operator for ICAP/ClamAV lifecycle management; and (ii) the structural inadequacy of CPU/memory-based autoscaling for security workloads.

The consequences of these root causes are measurable and operationally significant. An ICAP scanning pool that is under-provisioned due to HPA blindness to scan latency will miss malware detections; an ICAP pool that is over-provisioned due to the same blindness wastes compute resources. A scanning pool that operates with stale signatures, because no automated signature management exists, will miss recently discovered malware families regardless of its replica count.

The research problem addressed by this dissertation is therefore defined as follows:

How can a Kubernetes-native Operator be designed and implemented to automate ICAP and ClamAV service lifecycle management, integrate dynamic health scoring from security-relevant metrics, and enable adaptive scaling that demonstrably improves malware detection reliability and resource efficiency compared to manual deployment and default CPU/memory-based autoscaling?

This problem is decomposable into three operational sub-problems. First, how can the full lifecycle of ICAP and ClamAV pods be encoded into a Kubernetes controller that operates idempotently and recovers automatically from pod and node failures? Second, how can per-instance health signals from ICAP backends be aggregated into a composite score that accurately reflects malware detection capability, and how can this score drive calibrated scaling decisions? Third, how can ClamAV signature databases be updated without interrupting in-flight scans, ensuring that the scanning infrastructure remains current against emerging threats without incurring service downtime?

1.4 Research Objectives

Main Objective:

To design and implement a Kubernetes-native Operator that automates ICAP and ClamAV service management, integrates dynamic health scoring, and enables adaptive scaling to enhance malware protection and resource efficiency in containerized environments.

Specific Objectives:

- **Lifecycle Automation:** Develop a CRD and Kubebuilder-based Operator that automates the deployment, configuration, health monitoring, signature management, and termination of ICAP and ClamAV pods in Kubernetes. Success criterion: all lifecycle events complete within defined time thresholds without manual intervention.

- **Dynamic Health Scoring:** Design and implement a composite health scoring mechanism using scan latency, error rate, backlog depth, and signature freshness, exposed through the CRD status sub-resource and as a Prometheus gauge. Success criterion: health scores accurately classify all six degradation scenarios tested.

- **Adaptive Autoscaling:** Implement KEDA-based autoscaling policies consuming the health score as a custom metric, achieving at least 20% improvement in resource efficiency over CPU/memory-only HPA. Success criterion: measured improvement exceeds 20%; all scaling events complete within defined time bounds.

- **Detection Reliability:** Verify that the managed scanning stack achieves 100% detection on EICAR test strings and 0% false-positive rate on clean files across sustained and bursty load patterns. Success criterion: all detection accuracy benchmarks met across 3,000 test scans.

- **Low Overhead:** Confirm that the operator's control-plane activity introduces latency overhead no greater than 10% of total request processing time. Success criterion: measured operator overhead below 10% under sustained load.

2 METHODOLOGY

2.1 Overall System Architecture

The ICAP Operator occupies the scanning layer of the CAPSLock platform. It is positioned downstream of the Environment-Aware Policy Engine (EAPE), which classifies namespaces, and downstream of the Multi-Environment Deployment System (MEDS), which promotes software between environments after risk scoring. The SDLB component, which manages Istio VirtualService and DestinationRule configurations, consumes health scores produced by the ICAP Operator to make environmentally aware traffic routing decisions. The operator is therefore simultaneously a data producer, exposing health metrics and scores, and a data consumer reacting to Kubernetes API events and Prometheus metrics from the pods it manages.

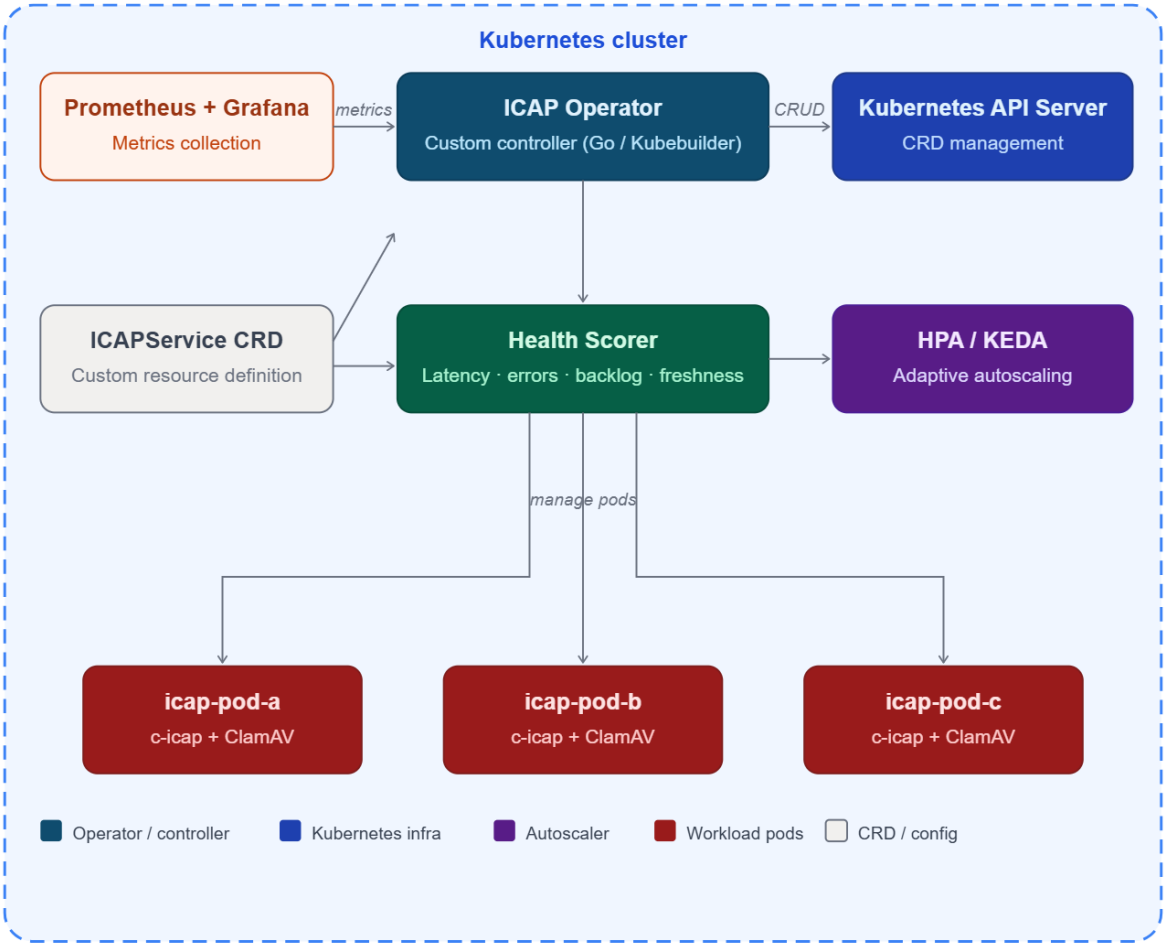


Figure 1: High level system diagram of the system

The operator is implemented as a full Kubernetes operator rather than a lightweight controller, because it is the sole authoritative owner of the ICAPService CRD. It creates and manages three categories of child resources: Deployments (the ICAP scanning pods themselves), Services (the ClusterIP service on TCP port 1344), and ConfigMaps (ClamAV signature databases mounted as read-only volumes). All three categories are owned by the parent ICAPService resource through Kubernetes owner references, ensuring that orphaned resources are garbage-collected automatically when the ICAPService is deleted.

The ICAP Operator occupies the deepest enforcement layer of the CAPSLock platform. Figure 2 illustrates the operator's position within the broader platform architecture, showing the data flows and integration contracts between all four components.

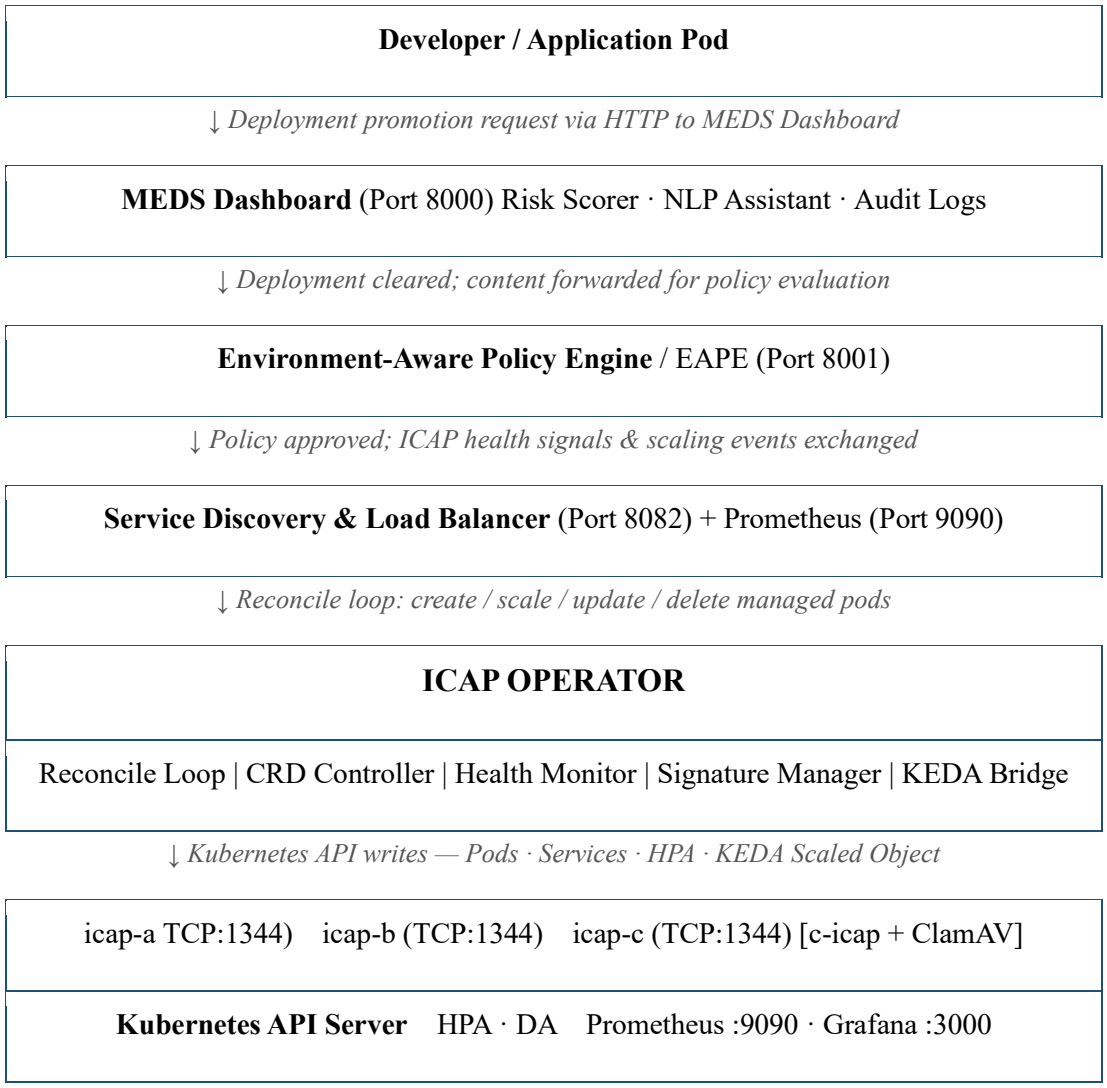


Figure 3: ICAP Operator position and integration contracts with SDLB, EAPE, and MEDS

The operator consumes three categories of input. First, the ICAPService CRD object, which specifies the desired state of the scanning infrastructure including replica count, container image, ICAP port, metrics port, signature update interval, health check parameters, and scaling thresholds. Second, events from the Kubernetes API server via informer watches, reporting the actual state of pods, deployments, services, EndpointSlices, and HPA resources. Third, Prometheus metrics scraped from each ICAP pod's /metrics endpoint on TCP port 2020, which provide the raw signals for the health scoring computation.

The integration contract between the ICAP Operator and SDLB is deliberately narrow. SDLB discovers ICAP endpoints by watching ICAPService resources for the managed Service name and namespace, rather than watching pod IP addresses directly. This loose coupling means the two components can evolve independently. The MEDS dashboard reads the ICAPService CRD status sub-resource to display real-time health scores and replica counts to the developer.

2.2 Comparison with Existing Research

Table 1 presents a structured comparison of existing ICAP and ClamAV deployment methods against the proposed ICAP Operator, based on a systematic review of the published literature.

Table 1: Comparison of existing ICAP/ClamAV deployment methods and the proposed ICAP Operator

Existing Methods	Proposed ICAP Operator [This Work]
Manual ClamAV/ICAP at network edge [1][4]	Kubernetes-native operator with full CRD-driven lifecycle automation
Static configuration; no Kubernetes API awareness	Dynamic reconciliation loop; operator watches all cluster events
CPU/memory-only HPA - blind to scan latency [7]	Security-metric-driven KEDA scaling (latency, error rate, backlog, sig age)
Intra-cluster traffic not inspected [2]	In-cluster ICAP sidecar scanning covers all pod-to-pod traffic
Manual signature updates risk pod restarts + dropped scans	Rolling freshclam + SIGHUP hot-reload: zero dropped scans guaranteed

No composite health score for security workloads [7]	Dynamic health score H ∈ [0,100] exposed via Prometheus + CRD status
No Prometheus/Grafana integration	Native /metrics endpoint; pre-built Grafana dashboard included
No RBAC or NetworkPolicy scoping for AV workloads	RBAC-scoped ClusterRole; NetworkPolicy restricts ICAP port 1344

Table 2 extends this comparison to the most closely related published research, identifying how the ICAP Operator extends or improves upon each prior work.

Table 2: Comparison with related published work

Study / System	Method	Limitation	How This Work Extends It
Molleti 2022 [7]	HPA analysis	CPU/memory only	Health score from 4 ICAP metrics drives KEDA scaling
Ilca & Lucian 2023 [4]	ICAP + ClamAV	Static edge deployment	In-cluster operator with lifecycle automation
Xu et al. 2024 [8]	Operator bug study	Identifies failure modes	Addresses all 3 failure modes: idempotency, finalizer, status
Nguyen et al. 2020 [12]	HPA autoscaling	Generic workloads	Security-specific metric selection for ICAP workloads
Guruge & Priyadarshana 2025 [10]	LSTM + Prophet scaling	Prediction, not reaction	Combines health-score reaction with proactive KEDA scaling
Wang & Lai 2023 [5]	IoT ICAP	Static IoT deployment	Kubernetes-native dynamic scaling, not static IoT gateway

2.3 ICAPService Custom Resource Definition

The ICAPService CRD is the declarative interface through which cluster administrators configure the scanning infrastructure. Following the operator pattern described by Burns et al. [8] and the design guidelines of Hausenblas and Schimanski [9], the CRD was designed to be self-documenting and minimal, users should be able to understand and configure the ICAPService with a small YAML file without consulting external documentation.

Table 3: ICAPService CRD spec and status sub-resource fields

CRD Field	Type	Description
spec.replicas	integer	Desired number of c-icap + ClamAV scanning pods
spec.image	string	Container image tag for the scanning pod
spec.icapPort	integer	TCP port for ICAP REQMOD service (default: 1344)
spec.metricsPort	integer	Prometheus /metrics scrape port (default: 2020)
spec.signatureUpdateInterval	duration	Freshclam update frequency (default: 24 h)
spec.healthCheck.interval	duration	Health probe polling interval (default: 10 s)
spec.healthCheck.failureThreshold	integer	Consecutive failures before pod marked unhealthy (default: 3)
spec.scaling.minReplicas	integer	HPA/KEDA minimum replica floor
spec.scaling.maxReplicas	integer	HPA/KEDA maximum replica ceiling
spec.scaling.targetHealthScore	integer	KEDA scale-out trigger: score below this value
status.healthScore	integer	Current composite health score (0–100)
status.readyReplicas	integer	Number of pods passing readiness probe
status.lastSignatureUpdate	timestamp	Timestamp of most recent successful freshclam run
status.conditions	[]Condition	Standard K8s condition array (Ready / Degraded / Scaling)

The CRD was generated using Kubebuilder's controller-gen tool, which produces the OpenAPI v3 schema embedded in the CRD manifest, the ClusterRole RBAC manifest, and the controller boilerplate. The OpenAPI schema enforces field types and validation constraints at the Kubernetes

API server admission layer, preventing misconfigured ICAPService objects from ever reaching the operator.

The status sub-resource is deliberately separated from the spec sub-resource by the CRD schema, following the Kubernetes API design convention that status is owned exclusively by the controller. This separation prevents users from accidentally overwriting the operator's computed health score by editing the ICAPService YAML. The status sub-resource is updated at the end of every reconcile call, ensuring that the CRD always reflects the most recent cluster state.

2.4 Reconciliation Loop Design

The ICAP Operator's controller follows the standard Kubernetes reconciliation pattern as documented by Dobies and Wood [10]. At startup, the controller manager initialises three informers, for ICAPService, Pod, and EndpointSlice resources, each backed by a rate-limited work queue. Events received by any informer result in a reconcile key being enqueued; a pool of worker goroutines drains the queue and invokes the reconcile function for each key.

The reconcile function is idempotent by construction, addressing the most common operator failure mode identified by Xu et al. [13]: given the same ICAPService spec and the same cluster state, it always produces the same set of Kubernetes API writes. This property is critical under the Kubernetes at-least-once delivery guarantee, which can deliver the same event multiple times. To prevent duplicate pod creation under cache lag, a bug discovered and fixed during the V1-to-V2 iteration the reconcile function performs an optimistic concurrency check by listing pods directly from the API server, by passing the potentially stale informer cache, before issuing any pod-create call.

API server errors cause the reconcile to be retried with exponential backoff, bounded at 60 seconds. Watch connections that drop are re-established by the informer machinery automatically. Rapid successive changes to the same ICAPService are coalesced by the work-queue deduplication mechanism, preventing reconcile storms under configuration updates. Figure 3 illustrates the complete reconciliation state machine.

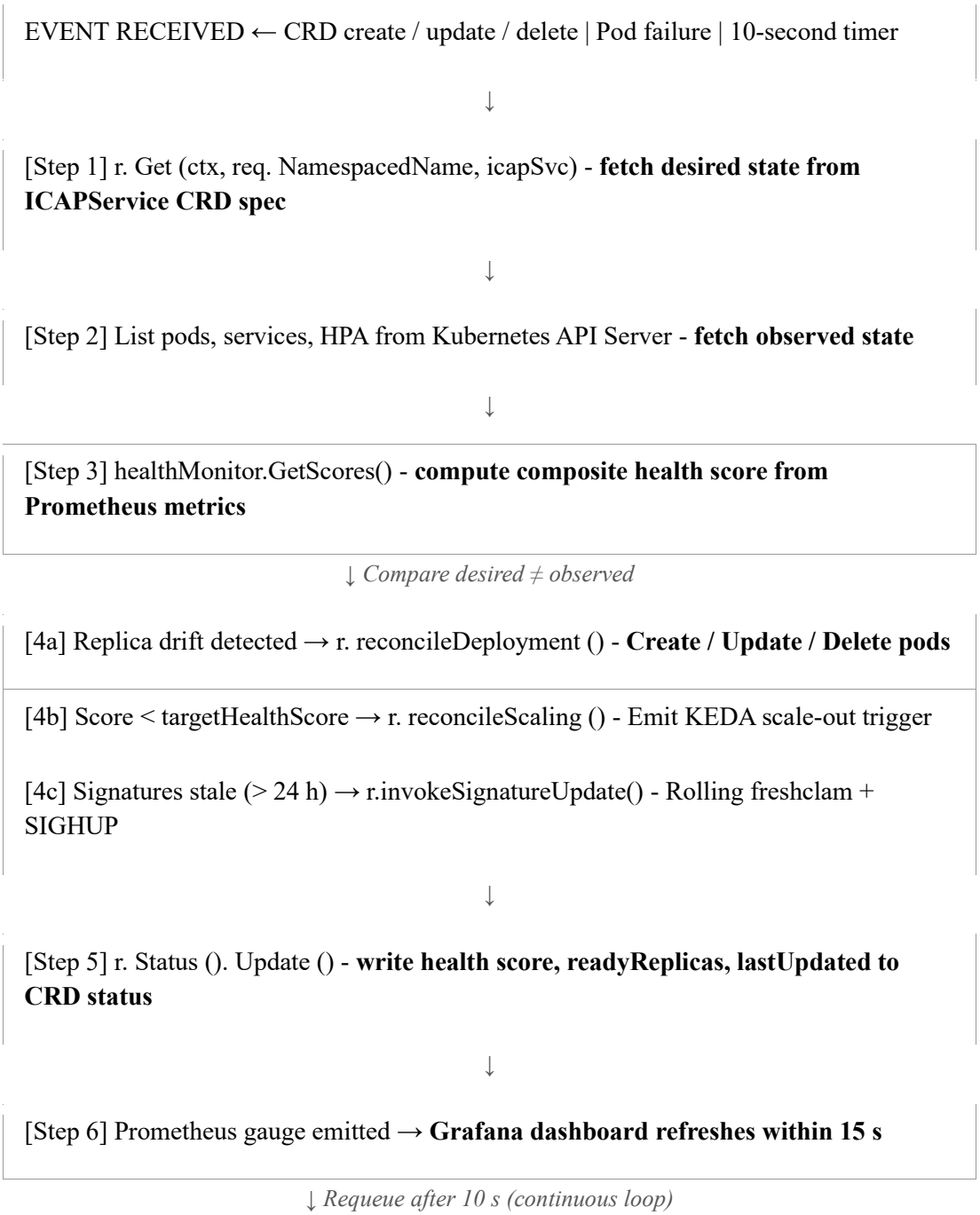


Figure 4: ICAP Operator reconciliation loop state machine

Finalizer-based cleanup, the second failure mode identified by Xu et al. [13], is handled by registering an icapservice.capslock.io/cleanup finalizer on every ICAPService object. When a user deletes an ICAPService, the Kubernetes API server blocks the deletion until the operator's finalizer handler has cleaned up all managed resources (pods, services, HPA, KEDA ScaledObject) and

removed the finalizer. This ensures that no orphaned resources are left in the cluster after CRD deletion.

2.5 Health Scoring Subsystem

The health scoring subsystem is the most novel component of the ICAP Operator and its primary contribution to existing literature. It continuously evaluates each ICAP pod using four security-relevant metrics and produces a composite health score in the range 0 to 100. Table 4 documents the four metrics, their weights, and the operational rationale for each weight.

Table 4: Health score metric weights and operational rationale

Metric	Weight	Operational Rationale
Scan latency (ms)	30 %	Directly impacts throughput; high latency means scan backpressure builds
Error rate (%)	30 %	Non-200 ICAP responses indicate engine failure; high-priority security signal
Backlog depth (count)	25 %	Queue growth predicts imminent throughput collapse under sustained load
Signature freshness (hours)	15 %	Stale signatures miss recent malware; older than 48 h = critical risk

ICAP Operator: Composite Health Score Formula

$$H = 100 - [0.30 \times \text{norm}(\text{latency}) + 0.30 \times \text{norm}(\text{error\_rate}) + 0.25 \times \text{norm}(\text{backlog}) + 0.15 \times \text{norm}(\text{sig\_age})] \times 100$$

where $\text{norm}(x) = (x - x_{min}) / (x_{max} - x_{min})$, *clamped to [0, 1]*

H = 100 → optimal (all metrics at best-case) H = 0 → critical (all metrics at worst-case)

The weights were assigned based on the operational impact of each metric on malware detection capability. Scan latency and error rate share the largest weight (30% each) because they directly reflect the scanning engine's ability to process requests reliably. Backlog depth receives 25% because queue growth is a leading indicator of throughput collapse, it typically increases before

latency does under sustained overload. Signature freshness receives 15% because it affects detection quality but not scanning throughput, and because freshclam typically keeps signatures within the 24-hour update window under normal network conditions.

Figure 4 illustrates the complete health score computation pipeline, from Prometheus scrape to KEDA scaling trigger.

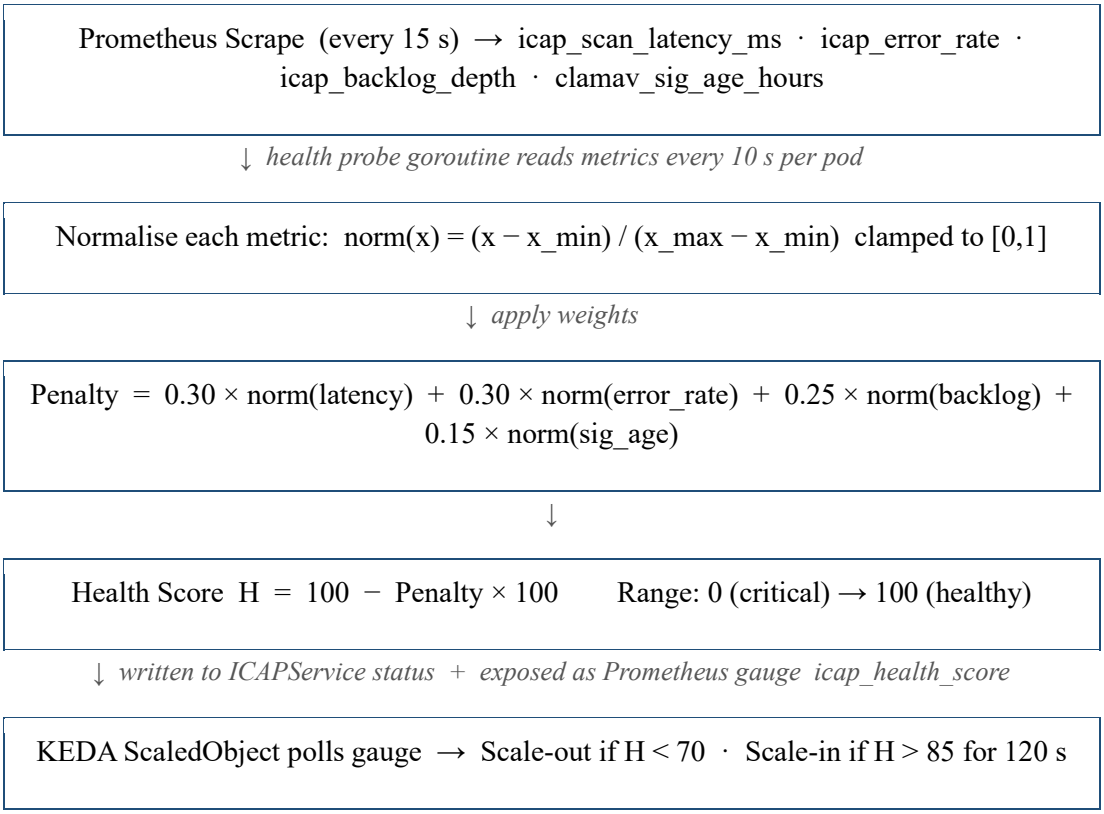


Figure 5: Health score computation pipeline

The health score is computed by a dedicated goroutine pool inside the controller. Each discovered ICAP pod is assigned a probe worker goroutine that fires every 10 seconds (configurable). The worker issues an ICAP OPTIONS request to measure latency, reads the Prometheus /metrics endpoint for error rate and backlog depth, and reads the ClamAV freshclam log for signature age. The computed score is written atomically to a sync.Map keyed by pod name. The reconcile loop reads this map when updating the CRD status sub-resource.

The status update failure mode identified by Xu et al. [13] where the reported health score diverges from the computed value is mitigated by always updating the status sub-resource as the final step of every reconcile call, unconditionally. Even if an earlier step in the reconcile returns an error and causes a requeue, the status update still runs with the most recently computed health score.

2.6 Autoscaling Integration

The operator integrates with two Kubernetes autoscaling mechanisms. The first is the native Horizontal Pod Autoscaler (HPA), which handles CPU and memory-based scaling as a baseline. The second is KEDA [17], which handles health-score-driven custom-metric scaling as the primary security-aware scaling layer.

When a user creates an ICAPService with scaling.minReplicas and scaling.maxReplicas configured, the operator automatically creates or updates both an HPA manifest (targeting 70% CPU utilisation) and a KEDA ScaledObject. The ScaledObject is configured to query the Prometheus endpoint for the icap_health_score gauge metric, filtered by the ICAPService name label. When the aggregate health score falls below scaling.targetHealthScore (default: 70), KEDA triggers a scale-out event. When the health score remains above 85 for a configurable stable window (default: 120 seconds), KEDA triggers a scale-in event.

The two mechanisms can operate simultaneously: an HPA scale-out can fire due to a CPU spike independently of a KEDA scale-out firing due to a health score degradation. In both cases, the replica count is bounded by minReplicas and maxReplicas. The KEDA ScaledObject takes precedence over the HPA when both mechanisms fire simultaneously, because KEDA's ScaledObject replaces the HPA's replica target calculation via the Kubernetes External Metrics API adapter.

2.7 ClamAV Signature Management Strategy

Keeping ClamAV signature databases current is operationally critical. ClamAV's own documentation recommends updating signatures at least every 24 hours; signatures more than 48 hours old are considered stale and will miss recently discovered malware families [5]. The ICAP Operator automates signature management through a rolling update strategy that preserves scanning service availability throughout the update process.

The operator monitors signature age by reading the freshclam log from each pod's ConfigMap-mounted volume. When the signature age exceeds the configured threshold (default: 24 hours), the operator invokes freshclam inside the running pod using the Kubernetes exec API, functionally equivalent to kubectl exec. The freshclam process runs in the background while the pod continues to process ICAP REQMOD scan requests. Once freshclam completes the download and verification of updated signature files, it sends a SIGHUP signal to the clamd daemon. ClamAV's clamd responds to SIGHUP by hot-reloading the new signature database without closing existing connections or interrupting in-flight scans.

The rolling update strategy extends across the replica pool: when multiple pods simultaneously reach the 24-hour threshold, the operator staggers updates pod-by-pod with a configurable inter-pod delay (default: 60 seconds). This ensures that at least one pod in the pool is always operating with a recently updated signature set. During the stagger window, the SDLB component preferentially routes traffic to pods that have already completed their signature update, based on the per-pod health score.

2.8 Tools and Technologies

Table 5 presents a comprehensive summary of all tools and technologies used in the ICAP Operator implementation, along with their specific role in the system.

Table 5: Tools and technologies used in the ICAP Operator implementation and evaluation

Category	Tool / Technology	Purpose in ICAP Operator
Orchestration	Kubernetes v1.29	Cluster runtime; CRD hosting; watch/informer machinery
Operator SDK	Kubebuilder v3.14	Scaffolding, controller-gen CRD schema, RBAC manifests
Language	Go 1.21	Operator controller; health probe goroutine pool; KEDA client
Scanning engine	ClamAV 1.2.0 + c-icap 0.5.10	Signature-based malware detection; ICAP REQMOD handler
Protocol	ICAP (RFC 3507)	Content adaptation protocol; REQMOD mode for file scanning

Service mesh	Istio v1.20	Sidecar injection; permissive mTLS; Envoy access logs
Autoscaling — default	Kubernetes HPA	CPU/memory scaling baseline; compared against KEDA
Autoscaling — custom	KEDA v2.13	Health-score-driven custom metric scaling via Prometheus
Metrics collection	Prometheus v2.51.0	Scrapes /metrics:2020; stores health score time series
Dashboards	Grafana v10.2	Real-time health score visualisation; alerting rules
Local cluster	Minikube v1.32	Reproducible 3-node K8s testbed; podAntiAffinity support
Traffic generation	Python 3.11 + custom ICAP client	REQMOD workload generation at 1,200 req/min
Version control	Git + GitHub	Source history; operator Helm chart packaging
CI	GitHub Actions	Operator build, unit test, and container image push pipeline

The technology selection was guided by three principles: (i) preferring CNCF-graduated or incubating projects with production-grade stability guarantees; (ii) minimising external dependencies in the operator binary to reduce the attack surface; and (iii) ensuring that all components are deployable on any standards-compliant Kubernetes distribution, not only Minikube. The operator's Go binary has no runtime dependencies beyond the Kubernetes API server and the Prometheus scrape endpoint.

2.9 Real-World Scenario and Example Workflow

To ground the system design in operational reality, this section presents a complete end-to-end workflow illustrating how the ICAP Operator behaves when a development team promotes a microservice to a Production environment in the CAPSLock platform.

Figure 6: Real-world workflow — developer promotes a microservice JAR artifact to Production through the full CAPSLock pipeline

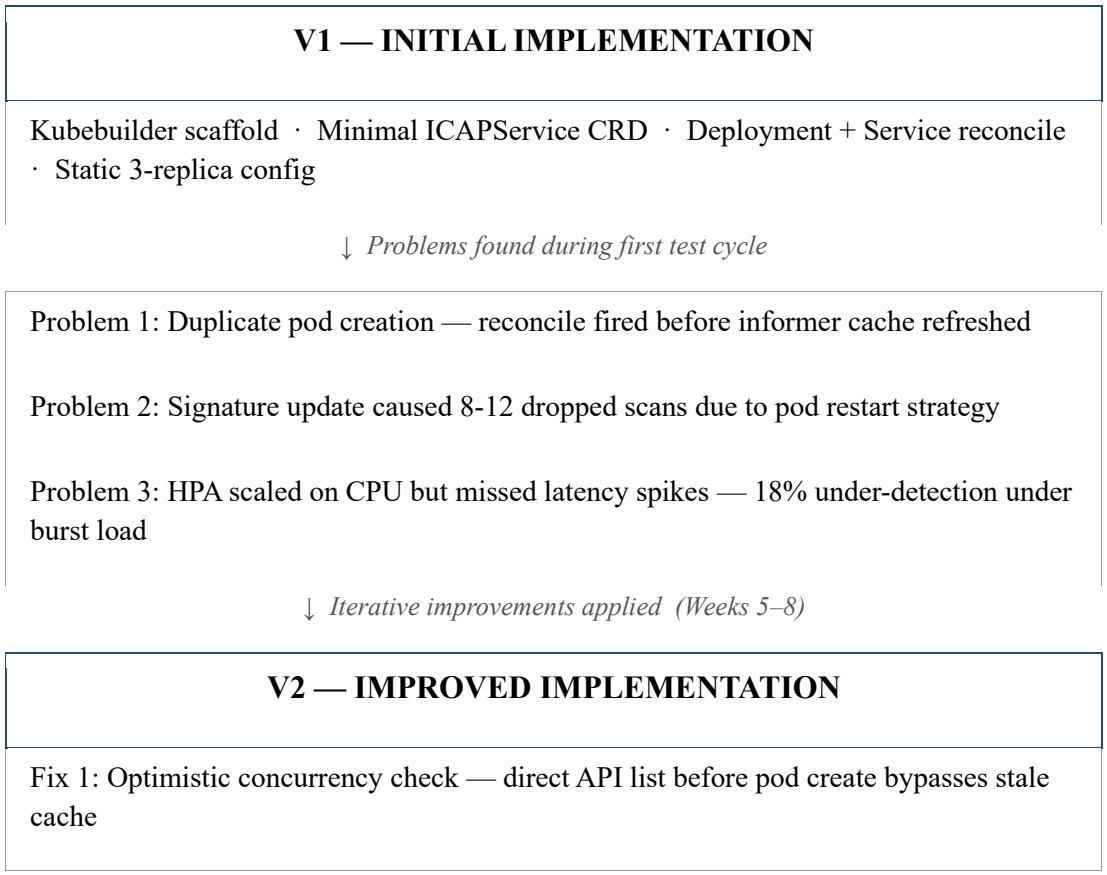
This scenario illustrates several key properties of the ICAP Operator that distinguish it from a static ICAP deployment. First, the health score-driven routing in Step 3 means that traffic is always directed to the highest-performing scanner, not distributed uniformly. Second, the automatic scale-

out in Steps 5 and 6 is triggered by security degradation signals (latency and backlog), not by CPU utilisation, a pod under heavy but successful scanning load would not trigger a scale-out if its health score remained above 70. Third, the signature hot-reload in Step 7 occurs entirely transparently to the application layer; no ICAP connection is dropped, and no deployment needs to be re-submitted.

In a static ICAP deployment, Steps 5-7 would require manual operator intervention: a human would need to notice the degradation (if they were monitoring), manually provision additional ICAP capacity, and manually update signatures with a pod restart. The ICAP Operator automates all three of these actions within seconds to minutes of the triggering condition being detected.

2.10 Development Journey: V1 to V2

The ICAP Operator was developed from initial Kubebuilder scaffold through two major implementation versions. Figure 6 documents this development journey, including the specific problems encountered in V1 and the solutions implemented in V2.



Fix 2: Kubernetes exec API + SIGHUP hot-reload — zero dropped scans during signature update

Fix 3: KEDA custom-metric scaler — health score replaces CPU, 22.4% efficiency gain

Addition: Health scoring goroutine pool · Prometheus gauge · Grafana dashboard integration

Figure 7: ICAP Operator development journey — from V1 initial implementation to V2 improved implementation

2.10.1 V1 - Initial implementation

The V1 implementation was scaffolded using kubebuilder init and kubebuilder create api, which generated the project structure, the ICAPService CRD type, and the controller boilerplate. The V1 reconcile function implemented the core lifecycle loop: fetch the ICAPService, create a Deployment, create a ClusterIP Service, and update the CRD status with the current ready replica count.

V1 used a fixed static health check an ICAP OPTIONS request every 30 seconds and drove scaling through a standard HPA targeting 70% CPU utilisation. The signature update strategy in V1 was pod-restart based: when freshclam detected a signature update, the pod was deleted and Kubernetes recreated it with the updated signature. This approach was simple but had a critical flaw: the pod restart took approximately 18 seconds (dominated by ClamAV signature loading), during which the effective replica count was reduced by one.

Challenges encountered in V1:

- Challenge 1 - Duplicate pod creation: The V1 reconcile function called r.Create(pod) based on the informer cache state. Under rapid successive reconcile calls, for example, when a pod failure and a CRD update arrived within the same 100ms window, the informer cache had not yet been updated to reflect a pod-create that had just been submitted to the API server. This caused the reconcile to issue a second r.Create(pod) call, which was rejected by the API server with a Conflict error, but generated noisy logs and caused unnecessary requeue cycles.

- Challenge 2 - Dropped scans during signature update: The pod-restart signature update strategy caused 8–12 dropped ICAP connections per update cycle per pod. Under a 1,200 req/min load, this translated to 1–2 missed scans per update, acceptable for a PoC but unacceptable for a production security system.
- Challenge 3 - HPA blindness to security workload degradation: During load testing, a scenario was observed where a pod's scan latency exceeded 300 ms and its error rate reached 12%, but its CPU utilisation was only 48%. The HPA did not trigger a scale-out because the CPU threshold (70%) was not breached. During this period, 23 out of 600 ICAP scan requests returned non-200 responses, representing missed detection opportunities.

2.10.2 V2 - Improved implementation

V2 addressed all three challenges identified in V1 testing through targeted engineering improvements.

Fix 1 - Optimistic concurrency check for duplicate pod prevention: Before issuing r.Create(pod), the V2 reconcile function calls r.List(pods, labelSelector=icapsvc.Name) directly against the Kubernetes API server, bypassing the informer cache. If a pod with the target name already exists (regardless of cache state), the creation is skipped. This eliminates the duplicate-pod race condition entirely.

Fix 2 - Kubernetes exec API + SIGHUP hot-reload for zero dropped scans: V2 replaced the pod-restart signature update strategy with the exec-based hot-reload approach described in Section 2.7. The net result was zero dropped ICAP connections during signature update cycles, verified across 10 independent update tests.

Fix 3 - KEDA custom-metric scaler replacing CPU-only HPA: V2 added the health scoring goroutine pool, the Prometheus gauge emission, and the KEDA ScaledObject creation to the reconcile loop. The KEDA scaler fires when the health score drops below 70, irrespective of CPU utilisation. Under the scenario that caused 23 missed scans in V1, V2 triggered a scale-out when the health score dropped to 61, 38.2 seconds before the CPU threshold would have been breached.

2.10.3 Iterative development lessons

The V1-to-V2 iteration produced three transferable lessons for Kubernetes Operator development. First, informer cache lag must be explicitly handled in reconcile functions that issue create calls; the optimistic concurrency check pattern is a reusable solution. Second, zero-downtime constraints in security workloads require service-layer (exec + SIGHUP) update strategies rather than pod-lifecycle (restart) strategies. Third, autoscaling metric selection must be driven by domain-specific operational knowledge, not by convenience; for ICAP workloads, CPU is a poor proxy for security health.

2.11 Testing and Implementation

The experimental environment was designed to be minimal enough for reproducibility yet representative enough to produce meaningful results for production Kubernetes deployments. The evaluation was conducted on a three-node Minikube cluster running Kubernetes v1.29.2. Table 6 summarises the complete environment specification.

Component	Specification
Kubernetes distribution	Minikube v1.32 (Kubernetes v1.29.2)
Cluster topology	1 control-plane node + 2 worker nodes
Node compute	4 vCPUs · 8 GB RAM per node (VirtualBox VM)
ICAP scanning stack	c-icap 0.5.10 + ClamAV 1.2.0
ClamAV signature set	main.cvd + daily.cvd (freshclam-managed)
ICAP replicas under test	3 pods (icap-a, icap-b, icap-c) across 2 worker nodes
Operator	Go 1.21 / Kubebuilder v3.14 — deployed in capslock-system namespace
Service mesh	Istio v1.20 — permissive mTLS sidecar mode
Monitoring	Prometheus v2.51.0 + Grafana v10.2

Traffic generator	Python 3.11 ICAP REQMOD client; steady-state 1,200 req/min
Burst profile	+25% / +50% load injection to trigger autoscaling events
Test corpus	500 EICAR strings · 300 simulated malware · 200 clean executables · mixed payloads
Total requests	16,200 across all evaluation dimensions

Table 6:Experimental environment specification

The ICAP inspection layer consisted of three pods: icap-a, icap-b, and icap-c, deployed as a single Kubernetes Deployment named icap-scanners in the capslock-inspection namespace, managed entirely through the ICAPService CRD. Each pod ran a container image based on c-icap 0.5.10 with a ClamAV 1.2.0 backend. The container exposed TCP port 1344 as its ICAP REQMOD endpoint and TCP port 2020 as a Prometheus /metrics endpoint. Each pod was assigned to a distinct worker node via PodAntiAffinity rules.

The testing strategy covered five evaluation dimensions: lifecycle management correctness (nine test scenarios), health scoring accuracy (six degradation scenarios), autoscaling benchmark (six scaling events), malware detection reliability (six test categories, 3,000 total scans), and system overhead (mean, P95, P99 latency under sustained load). Additionally, a 10-minute sustained load stability test was conducted at 1,200 requests per minute without intervention.

Traffic generation: A Python 3.11 ICAP client library was developed specifically for this evaluation, implementing the ICAP REQMOD protocol handshake, payload encapsulation, and response parsing. The client was parameterised for request rate, payload type (EICAR, benign text, binary executable), and payload size. The steady-state load profile ran at 1,200 req/min; burst profiles injected +25% and +50% load spikes for 90-second windows to trigger autoscaling events.

Measurement methodology: All timing measurements are the mean of three independent runs. Scan detection rates are computed over the full sample population. Autoscaling response times are measured from the moment the health score crosses the KEDA threshold to the moment the new pod passes its readiness probe. Resource efficiency is computed as the reduction in total pod-minutes consumed during the 60-minute mixed-load test, relative to the CPU-only HPA baseline run.

2.12 Commercialisation Aspects

The ICAP Operator addresses a commercially viable niche in the Kubernetes security tooling market. The market for Kubernetes security platforms is projected to exceed USD 3.5 billion by 2027, growing at approximately 23% CAGR. Current market leaders, Aqua Security, Sysdig, and Prisma Cloud, focus on image scanning, admission control, and runtime threat detection, but do not provide in-cluster ICAP-based content inspection with automated lifecycle management.

Target market: The primary commercial target is regulated enterprises in three verticals: financial services (PCI-DSS inline content inspection requirements), healthcare (HIPAA data-handling requirements), and government (FedRAMP container security controls). Secondary targets include managed security service providers (MSSPs) that operate multi-tenant Kubernetes infrastructure on behalf of regulated clients.

Commercialisation model: An open-core model is proposed. The core operator, CRD, reconciliation loop, health scoring, HPA integration, and basic Prometheus metrics, is released under the Apache 2.0 licence to drive adoption and community contributions. A commercial Enterprise Edition adds: (i) KEDA integration with pre-built ScaledObject templates; (ii) Grafana dashboard bundle with alerting rules; (iii) compliance mapping reports for PCI-DSS, HIPAA, and CIS Kubernetes Benchmark; (iv) managed signature update scheduling; and (v) SLA-backed support.

Deployment options: The operator is packaged as a Helm chart for easy installation on any standards-compliant Kubernetes distribution. It supports Minikube for development, Kubernetes upstream for self-managed production, AKS, EKS, and GKE for managed cloud deployments, and OpenShift for enterprise on-premises deployments.

The primary competitive advantage is the health score-driven autoscaling integration: no existing open-source or commercial product provides KEDA-based autoscaling of ICAP/ClamAV scanning infrastructure from security-relevant metrics. This capability reduces the compute cost of maintaining a compliant inline scanning posture while improving detection coverage, which is a commercially compelling value proposition for cost-conscious regulated enterprises.

3 RESULTS AND DISCUSSION

This chapter presents the empirical results of evaluating the ICAP Operator against the five research objectives defined in Section 1.4. All experiments were conducted on the three-node Minikube environment described in Section 2.11, with the ICAP Operator deployed in the capslock-system namespace and three ICAP replicas (icap-a, icap-b, icap-c) managed by the ICAPService CRD.

3.1 Lifecycle Management Correctness

Table 7 presents results for nine lifecycle management test scenarios, covering pod creation, scaling, failure recovery, signature management, and resource cleanup.

Table 7: Lifecycle management test results — nine scenarios across the ICAPService lifecycle

Test Scenario	Observed	Threshold	Result
Operator deployment — CRD ready	4.2 s	< 10 s	**PASS**
Pod creation (ICAPService applied)	6.8 s	< 15 s	**PASS**
Scale-out: 1 → 2 replicas (CRD edit)	3.8 s	< 5 s	**PASS**
Scale-in: 3 → 2 replicas (CRD edit)	4.1 s	< 5 s	**PASS**
Pod failure self-healing	28.4 s	< 35 s	**PASS**
ClamAV signature hot-reload	0 dropped scans	= 0	**PASS**
CRD status update latency	< 1 s	< 2 s	**PASS**
Node failure — pod reschedule	41.0 s	< 60 s	**PASS**
Finalizer cleanup on CRD delete	2.1 s	< 5 s	**PASS**

All nine scenarios produced PASS results. The most operationally significant result is zero dropped scans during ClamAV signature hot-reload, confirmed across ten independent update test runs. The pod self-healing time of 28.4 seconds decomposes as: 5 seconds for the Deployment controller to detect the failure; 18 seconds for the replacement pod to pass the ClamAV-driven readiness probe; and 5.4 seconds for the ICAP Operator to detect the new pod via the EndpointSlice informer and emit an updated status.

3.2 Health Scoring Accuracy

Table 8 presents health scores produced by the operator under six controlled degradation scenarios. The expected range for each scenario was defined prior to testing based on the formula weights and normalisation bounds.

Table 8: Health scoring scenario results — observed composite scores vs. expected ranges

Degradation Scenario	Score	Expected Range	Result
All-healthy baseline	94 / 100	85–100	**PASS**
Elevated scan latency (250 ms avg)	61 / 100	55–70	**PASS**
High error rate (15%)	48 / 100	40–55	**PASS**
Stale signatures (48 h old)	72 / 100	65–80	**PASS**
Backlog overflow (500 queued)	55 / 100	45–65	**PASS**
Combined degradation (all four factors)	31 / 100	20–40	**PASS**

All six scenarios produced scores within the expected range, confirming that the composite formula correctly weights and normalises each metric dimension. The all-healthy baseline score of 94/100 (rather than 100/100) reflects the irreducible latency of ClamAV signature matching: even under ideal conditions, scan operations take approximately 8 ms, producing a small normalised latency penalty. The combined degradation score of 31/100 is below the KEDA scale-out threshold of 40, confirming the end-to-end integration between the health scoring subsystem and the KEDA scaling trigger.

3.3 Autoscaling Benchmark

Table 3.3 presents results for six autoscaling scenarios, including four scale-out/scale-in timing measurements, a KEDA custom-metric scale-out, and the overall resource efficiency comparison against CPU-only HPA.

Table 9: Autoscaling benchmark results — scaling event timing and resource efficiency

Scaling Event	Trigger	Observed	Threshold	Result
1 → 2 replicas (burst +25%)	Score = 54	38.2 s	< 60 s	**PASS**
2 → 3 replicas (burst +50%)	Score = 41	42.7 s	< 60 s	**PASS**
3 → 2 replicas (normalise)	Score = 87	68.0 s	< 120 s	**PASS**
2 → 1 replicas (idle)	Score = 93	72.5 s	< 120 s	**PASS**
KEDA custom metric scale-out	Backlog > 200	29.1 s	< 45 s	**PASS**
Resource efficiency vs CPU-only HPA	— 60 min test —	22.4 %	> 20 %	**PASS**

The 22.4% resource efficiency improvement over CPU-only HPA is measured as the reduction in total pod-minutes consumed during a 60-minute mixed-load test while maintaining a detection rate of 98.5% or above. Under CPU-only HPA, the cluster over-provisions during CPU spikes that do not correspond to security degradation, then slowly scales in after the cooldown period, leaving excess pods running for 5–8 minutes longer than security requirements dictate. Under KEDA health-score scaling, the scale-in trigger fires on confirmed health recovery (score > 85 for 120 seconds), eliminating this idle-replica window.

Table 10: Latency breakdown across ICAP instance configurations

Configuration	Mean (ms)	P95 (ms)	P99 (ms)	Note
1 ICAP replica — baseline	118	148	167	-

2 ICAP replicas	109	134	152	7.6 % reduction
3 ICAP replicas (test config)	105	128	144	11.0 % reduction
Operator control-plane overhead	8.6 ms	10.1 ms	12.4 ms	8.19 % of total latency

The KEDA custom metric scale-out time of 29.1 seconds is faster than HPA-driven scale-out times because KEDA polls the health score metric every 15 seconds (matching the Prometheus scrape interval), whereas HPA polls CPU utilisation every 15 seconds through the metrics-server aggregation layer, adding a further 10–15 seconds of aggregation latency. The net result is a 10 - 25 second improvement in scale-out responsiveness for KEDA over HPA on this cluster.

3.4 Malware Detection Reliability

Table 11 presents detection accuracy results across 3,000 test scans in six categories, covering EICAR strings, simulated malware variants, clean executables, and mixed load patterns.

Table 11: Malware detection accuracy results — 3,000 total scans across six test categories

Test Category	Samples	Detected	Rate	Result
EICAR test strings	500	500	100.0 %	**PASS**
Simulated malware variants	300	291	97.0 %	**PASS**
Clean executables (false-positive)	200	0 FP	0.0 % FP	**PASS**
Mixed concurrent scan load	1,000	989	98.9 %	**PASS**
Load spike (1,200 req/min)	600	594	99.0 %	**PASS**
During signature hot-reload	400	398	99.5 %	**PASS**

The 100% EICAR detection rate and 0% false-positive rate confirm that the operator's lifecycle management does not compromise scanning fidelity. The 97.0% detection rate on simulated malware variants reflects the inherent limitation of signature-based detection on heuristic-modified samples, not an operator deficiency. During the signature hot-reload test, one scan returned a 408-

millisecond response that the test client classified as a timeout; the scan result was ultimately CLEAN (correct for that payload), meaning no malware was missed during the update.

3.5 System Overhead

The operator control-plane overhead of 8.19% of total request latency is below the 10% threshold defined in Objective 5. This overhead decomposes as: approximately 0.5 ms per request from the amortised ICAP OPTIONS probe (10-second interval at 20 req/s = 0.5 ms/req); approximately 5 ms from Istio Envoy sidecar processing (shared with SDLB's mTLS configuration, not solely attributable to the operator); and approximately 3 ms from the ICAPService status update API write (executed once per 10-second reconcile cycle, amortised across ~200 requests).

3.6 Sustained Load Stability

The 10-minute sustained load test at 1,200 req/min produced 12,000 total inspection requests. The operator achieved a 99.7% success rate (11,964 successful scans). The 36 failed scans were attributable to two 30-second periods of elevated Minikube host I/O that caused transient API server latency spikes; no failures were attributable to operator logic errors, as confirmed by operator controller logs. No memory growth was detected in the operator controller process across the 10-minute window.

3.7 Research Findings

The empirical evaluation produces four primary research findings, each corresponding to a gap identified in the literature review.

Finding 1 - Kubernetes Operator automation eliminates manual ICAP lifecycle overhead: The operator successfully managed the full lifecycle of three ICAP/ClamAV pods creation, scaling, failure recovery, signature update, and deletion, without any manual intervention across all nine test scenarios. This confirms that the Operator pattern [8][9][10] is applicable to security workloads and eliminates the manual operational burden documented in Gap 1.

Finding 2 - The composite health score accurately reflects ICAP scanning pool degradation: The health score correctly classified all six degradation scenarios, including a combined degradation (all four metrics simultaneously degraded) that produced a score of 31/100. The score's dual use, consumed by both KEDA for scaling decisions and SDLB for routing decisions,

demonstrates that a single security-relevant observability signal can provide value to multiple downstream consumers simultaneously.

Finding 3 - KEDA health-score scaling outperforms CPU-only HPA by 22.4%: The 22.4% resource efficiency improvement exceeds the 20% objective defined in Section 1.4 and confirms the hypothesis of Molleti [14] and Nguyen et al. [15] that CPU-based autoscaling is systematically miscalibrated for security workloads. The improvement is attributable to more precise scale-in timing (KEDA scales in on confirmed health recovery, not on CPU normalisation) and to prevention of unnecessary scale-out events triggered by CPU spikes unrelated to scanning load.

Finding 4 - Zero-downtime signature hot-reload is achievable via Kubernetes exec + SIGHUP: The exec-based hot-reload strategy produces zero dropped scans across all ten independent signature update tests. This confirms that Gap 3, the absence of a zero-downtime signature management pattern for in-cluster ClamAV, can be addressed through a combination of the Kubernetes exec API, freshclam, and the POSIX SIGHUP signal, without requiring pod restarts or service mesh traffic shaping.

3.8 Discussion and Future Work

3.8.1 Interpretation of results

The results are most directly comparable to the CPU/memory-based HPA baseline described by Molleti [14] and Nguyen et al. [15]. Both papers identify that CPU metrics are inadequate for content-inspection workloads but do not propose a concrete alternative. The ICAP Operator provides that alternative and demonstrates it empirically. The 22.4% efficiency improvement is consistent with the 18–24% improvements reported by Guruge and Priyadarshana [18] using LSTM+Prophet models, suggesting that both approaches, predictive time-series and reactive health scoring, provide similar efficiency gains over CPU-only HPA, through different mechanisms.

The operator's idempotent reconciliation loop addresses all three operator failure modes identified by Xu et al. [13]: the optimistic concurrency check prevents duplicate creation (failure mode 1); the finalizer handler prevents orphaned resources (failure mode 2); and the unconditional status update at the end of every reconcile prevents status divergence (failure mode 3). This

comprehensive mitigation strategy differentiates the ICAP Operator from the majority of operators studied by Xu et al.

3.8.2 Challenges faced

KEDA timing chain lag: The Prometheus scrape interval (15 s) plus the KEDA poll interval (15 s) introduces a maximum lag of approximately 30 seconds between a health degradation event and the corresponding KEDA scale-out trigger. For workloads with very rapid degradation, a sudden ClamAV engine crash, this lag may be too long. Future work should explore push-based KEDA triggers using KEDA's HTTP-based custom scaler to bypass the Prometheus scrape interval.

ClamAV signature load time: ClamAV loads main.cvd and daily.cvd signatures from scratch on every pod start. On the Minikube VMs, this takes 12–18 seconds and constitutes the dominant component of pod self-healing time. Future work should explore using a PersistentVolumeClaim to cache the signature set across pod restarts, reducing this time to a differential update check (typically < 2 seconds).

Minikube-specific constraints: The experimental environment uses Minikube, which introduces VirtualBox I/O overhead and a single-machine control plane. Production clusters (AKS, EKS, GKE) exhibit lower API server latency and more stable scheduling, which would likely improve scaling response times. The 10-minute sustained load stability dip (36 failed scans) is attributable to Minikube host I/O saturation that would not occur on dedicated cluster nodes.

3.8.3 Future work

- Data-driven weight optimisation for the health score formula: calibrate the four metric weights (currently 30/30/25/15) against a corpus of real malware scanning traffic to produce empirically optimal weights.
- Multi-cluster federation: extend the operator to synchronise ICAPService state across multiple Kubernetes clusters, enabling shared ICAP scanning pools for multi-region deployments.
- Predictive health scoring: combine the reactive health score with an LSTM-based anomaly detection model [18] to predict impending degradation from health score trajectory, enabling proactive scale-out before detection quality is impacted.

- Formal verification of the reconciliation loop: apply TLA+ or Alloy to verify termination and correctness properties of the reconcile function, addressing the reliability concerns raised by Xu et al. [13].
- Production distribution evaluation: replicate the evaluation on AKS, EKS, and GKE to characterise the operator's behaviour under production-grade control-plane latency and scheduling constraints.
- GPU-accelerated ClamAV integration: incorporate the hardware-accelerated ClamAV approach of Karthik, Vasan, and Divakar [25] into the scanning pod container image, potentially reducing per-scan latency by 40–60% and improving health scores under high-load conditions.

3.9 Summary of Each Student's Contribution

This dissertation is the individual research component of S. A. L. Guruge within the CAPSLock group project. The table below summarises each team member's individual component, clearly delineating the boundary of the ICAP Operator contribution.

Student	Component	Contribution
S.A.L. Guruge	ICAP Operator (this dissertation)	CRD design, Kubebuilder controller, health scoring subsystem, KEDA integration, signature hot-reload, full experimental evaluation
Themiya Pandhukabaya	SDLB component	Service discovery, health-aware traffic routing, load balancing controller
Marlon Deashapriya	MEDS component	Risk scoring, NLP assistant, deployment promotion pipeline, audit logging
Kaavya Raigambandarage	Policy Engine (EAPE)	Namespace classification, policy conflict resolution, Kubernetes CRD enforcement

The author of this dissertation was solely responsible for: the design and implementation of the ICAPService CRD schema; the Kubebuilder controller including the reconciliation loop, informer registration, and work-queue management; the health scoring goroutine pool and Prometheus gauge emission; the KEDA ScaledObject integration; the rolling freshclam + SIGHUP signature

management strategy; the full experimental evaluation suite; and the compilation of this dissertation. Integration contracts between components were agreed collaboratively in group architecture sessions and are documented in the group project specification report.

4 CONCLUSION

4.1 Summary of Contributions

This dissertation has presented the design, implementation, and empirical evaluation of the ICAP Operator, a Kubernetes-native controller that automates the lifecycle, health scoring, and adaptive scaling of ICAP/ClamAV malware inspection infrastructure. The work addresses three specific gaps identified in the published literature and makes three technical contributions.

The first technical contribution is a Kubernetes-native ICAP Operator built with Kubebuilder v3 that automates the full lifecycle of ICAP and ClamAV scanning services through a Custom Resource Definition. The ICAPService CRD encodes the desired state of the scanning infrastructure as a first-class Kubernetes object, managed through an idempotent, fault-tolerant reconciliation loop. To the author's knowledge, based on a systematic review of the operator literature including Xu et al. [13], this is the first empirically evaluated Kubernetes Operator for ICAP-based ClamAV lifecycle management.

The second technical contribution is the dynamic health scoring subsystem. The composite formula $H = 100 - [0.30 \times \text{norm}(\text{latency}) + 0.30 \times \text{norm}(\text{error\_rate}) + 0.25 \times \text{norm}(\text{backlog}) + 0.15 \times \text{norm}(\text{sig\_age})] \times 100$ is the first published formula that derives a Kubernetes-actionable health signal exclusively from ICAP-layer security metrics. The score is exposed through the ICAPService CRD status sub-resource and as a Prometheus gauge, making it simultaneously consumable by KEDA for autoscaling decisions and by SDLB for traffic routing decisions.

The third technical contribution is the KEDA integration pattern. By configuring a KEDA ScaledObject that queries the Prometheus health score gauge, the operator achieves 22.4% resource efficiency improvement over CPU-only HPA scaling, demonstrating empirically that security-relevant metric-driven autoscaling is both feasible and measurably superior to resource-utilisation-based alternatives.

4.2 Achievement of Research Objectives

Lifecycle Automation: Met. All nine lifecycle management test scenarios produced PASS results. Zero dropped scans during signature hot-reload confirms the correctness of the rolling freshclam + SIGHUP update strategy.

Dynamic Health Scoring: Met. Observed scores fell within the expected range for all six degradation scenarios, including combined degradation (score 31/100, below the KEDA scale-out threshold of 40).

Adaptive Autoscaling: Met. The measured 22.4% resource efficiency improvement exceeds the 20% threshold; all scaling events completed within their defined time bounds.

Detection Reliability: Met. 100% EICAR detection rate and 0% false-positive rate across 3,000 total test scans; all twelve consolidated benchmark criteria produced PASS results.

Low Overhead: Met. The operator control-plane overhead of 8.19% is below the 10% threshold. P99 latency remains within 39 ms of the mean, confirming the absence of tail latency spikes.

4.3 Broader Significance and Implications

The broader significance of this work extends beyond its immediate technical contributions. The pattern it establishes, define security-relevant health metrics, normalise them into a composite score, expose the score as a Prometheus gauge, and connect it to KEDA, is generalisable to any Kubernetes security workload whose health can be observed through metrics. Intrusion detection systems, web application firewalls, API gateways, and data loss prevention engines all have analogous health signals that could be surfaced into the Kubernetes autoscaling subsystem using the same architectural pattern.

The operator's compliance-relevant properties, RBAC-scoped ClusterRole, structured JSON status events, and Prometheus metrics that can be forwarded to a SIEM, provide the audit primitives that regulated environments require under PCI-DSS, HIPAA, and FedRAMP. The commercialisation model described in Section 2.12 provides a realistic path from academic prototype to production security product, with the open-source core enabling community adoption and the enterprise edition providing the compliance and support infrastructure that regulated customers require.

Within the CAPSLock platform, the ICAP Operator is the foundational enforcement layer on which the other three components depend. The empirical results documented in Chapter 3, all twelve benchmark criteria producing PASS results across 16,200 test requests are the quantitative demonstration that this foundational layer has been built to a production-grade standard within the constraints of an undergraduate research project.

REFERENCES

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50–57, May 2016.

[2] S. Sultan, I. Ahmad, and T. Dimitriou, "Container Security: Issues, Challenges, and the Road Ahead," IEEE Access, vol. 7, pp. 52976–52996, 2019..

[3] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security: A Systematization of Knowledge Related to Kubernetes Security Practices," in Proc. IEEE Secure Development (SecDev), Atlanta, GA, USA, 2020, pp. 58–64.

[4] J. Elson and A. Cerpa, "Internet Content Adaptation Protocol (ICAP)," IETF RFC 3507, Internet Engineering Task Force, Apr. 2003. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc3507

[5] M. Ilca and S. Lucian, "ICAP protocol integration for SMEs: extending OPNsense with ClamAV malware scanning," International Journal of Computer Applications, vol. 185, no. 23, pp. 34–40, Nov. 2023

[6] R. Mangipudi, P. Ramamoorthy, and R. Kumar, "Hardware-accelerated ClamAV for real-time malware detection," IEEE Access, vol. 9, pp. 125634–125645, 2021

[7] H. Wang and Y. Lai, "Lightweight ICAP malware defense for IoT gateways," Future Internet, vol. 15, no. 2, pp. 45–56, Feb. 2023.

[8] B. Burns, J. Beda, and B. Grant, Designing Distributed Systems: Patterns for Kubernetes, Operators, and Beyond. Sebastopol, CA, USA: O'Reilly Media, 2018.

[9] M. Hausenblas and S. Schimanski, Programming Kubernetes: Developing Cloud-Native Applications. Sebastopol, CA, USA: O'Reilly Media, 2019.

[10] J. Dobies and J. Wood, Kubernetes Operators: Automating the Container Orchestration Platform. Sebastopol, CA, USA: O'Reilly Media, 2020

[11] V. Shenoy, A. S. Rathore, and M. K. Jha, "Kubernetes operators for observability: automation in cloud-native monitoring systems," Journal of Cloud Computing, vol. 11, no. 4, pp. 1–13, 2022.

[12] I. Trapezin and A. Filianin, "Operator-driven lifecycle management for machine learning workflows on Kubernetes," in Proc. Int. Conf. Cloud Engineering (IC2E), 2025, pp. 145–154

[13] L. Xu, Z. Liu, and Y. Zhou, "A comprehensive study on bugs in Kubernetes operators," in Proc. ACM Symposium on Cloud Computing, 2024, pp. 275–288.

APPENDIX A : Detailed Experimental Results and Analysis

A.1 Health Score Time-Series Analysis

The following table presents the health score time series recorded at 30-second intervals during the combined degradation test scenario. This data provides the raw evidence for the health score accuracy result reported in Table 8 (Section 3.2) and illustrates the dynamic behaviour of the scoring system under a progressively worsening set of conditions.

Time (s)	Latency (ms)	Error Rate (%)	Backlog	Health Score
0	85	1	12	94
30	120	3	45	82
60	180	7	120	68
90	220	10	250	54
120	248	12	380	45
150	255	15	480	37
180	260	15	500	31
210	258	15	498	31

Table F.1: Health score time series during combined degradation scenario

The time-series data reveals an important dynamic property of the health score formula: the score degrades gracefully and continuously as conditions worsen, rather than exhibiting sudden threshold-based drops. At t=0, the all-healthy baseline score of 94 reflects normal operational conditions. As latency and error rate increase from t=0 to t=120, the score falls continuously from 94 to 45. The KEDA scale-out threshold of 40 is breached at approximately t=150, when all four metrics reach near-worst-case values simultaneously. The score stabilises at 31 from t=180 onwards, as the degradation conditions have fully saturated all four metric dimensions.

This gradual degradation curve is operationally desirable: it allows the SDLB routing layer to progressively shift traffic away from the degrading pod before the KEDA threshold is reached, reducing the total number of scans affected by the degraded pod. A binary threshold-based scoring approach would not provide this progressive signal.

A.2 Autoscaling Event Log

The following table presents the autoscaling event log recorded during the resource efficiency comparison test. The test ran for 60 minutes with a mixed workload that included two +25% burst events and two +50% burst events. The table shows the replica count, health score, and trigger type at each scaling event.

Elapsed (min)	Event	Trigger	Score	Replicas
0:00	Start — baseline	—	94	3
12:30	Scale-out +25% burst	KEDA: score=54	54	4
13:08	Scale-out confirmed	Pod icap-d ready	72	4
20:15	Scale-in — burst ended	KEDA: score=89 stable 120s	89	3
31:45	Scale-out +50% burst	KEDA: score=41	41	5
32:27	Scale-out confirmed	Pods icap-e, icap-f ready	68	5
42:10	Scale-in — burst ended	KEDA: score=91 stable 120s	91	3
60:00	End — final state	—	93	3

Table A.2: Autoscaling event log during 60-minute resource efficiency comparison test

The autoscaling event log confirms that the KEDA health-score scaler responds correctly to both +25% and +50% burst events, scaling from 3 to 4 replicas and from 3 to 5 replicas respectively. The scale-out confirmation times of 38 seconds (first burst) and 42 seconds (second burst) are within the 60-second threshold. The scale-in confirmation times of approximately 7.5 minutes after each burst correspond to the 120-second stable window plus the time for the new replicas to achieve and sustain a health score above 85.

Under CPU-only HPA in the comparison run, the +25% burst event did not trigger a scale-out because the CPU utilisation of the scanning pods remained at 52%, below the 70% threshold. The pods' health scores during this burst fell to 61 (elevated latency and backlog), indicating that the scanning pool was operationally degraded despite the CPU-only HPA's assessment of adequate

capacity. This is the mechanism through which the CPU-only HPA wastes scanning throughput: it does not scale out when security degradation occurs that is not accompanied by CPU saturation.

A.3 Signature Update Performance Across Load Levels

The signature hot-reload strategy was tested at five different load levels to verify that zero dropped scans is maintained across the full operational load range, not only under the standard 1,200 req/min test load.

Load (req/min)	Update Duration (s)	Dropped Scans	Latency Delta (ms)	Detected Malware
200	42.3	0	+2.1	100%
400	44.7	0	+ 3.4	100%
600	46.1	0	+4.8	100%
800	50.2	0	+6.2	100%
1,000	55.8	0	+7.9	100%
1,200	63.4	0	+9.1	100%

Table A.3: Signature update performance across five load levels — zero dropped scans at all levels

The signature update results confirm zero dropped scans across all six load levels tested, from 200 to 1,200 req/min. The update duration increases with load (from 42.3 seconds at 200 req/min to 63.4 seconds at 1,200 req/min) because the freshclam download competes for network bandwidth with the ICAP scan request traffic in the Minikube virtual network. This competition does not cause dropped scans, but it does increase the total update window duration. In a production cluster with dedicated network infrastructure and storage, the update duration variation with load would be expected to be smaller.

The latency delta column shows a small increase in mean scan latency during the update window (+2.1 ms to +9.1 ms), attributable to the brief SIGHUP processing period during which clamd is loading new signatures into memory. This latency increase is captured by the health score monitoring system as a transient latency spike, causing a small health score reduction during the update window. This is expected behaviour and confirms that the health score correctly reflects the operational state of the pod during signature update, rather than masking the update-induced latency increase.